COMS 3110: Homework 1
Due: September 19th, 11:59pm
Total Points: 50

Late submission policy. An assignment that is submitted one day late will get a penalty of 10%. An assignment that is submitted two days late will get a penalty of 20%. Assignments submitted after two days will not be graded and will get no points. For example, if an assignment is due on Friday by midnight, a submission on Saturday will be penalized by 10%, and a submission on Sunday will be penalized by 20%. A submission on Monday will get no points.

Submission format. Homework solutions must be **handwritten**. You may either write directly on a tablet or write on paper and scan your work. In both cases, the final submission must be compiled into a **single PDF file**. Typed solutions will only be accepted with prior, explicit permission from the instructor. We reserve the right **NOT** to grade homework that does not follow these requirements. Name your submission file: `<Your-net-id>-3110-hw1.pdf`. For instance, if your netid is `asterix`, then your submission file will be named `asterix-3110-hw1.pdf`. Each student must hand in their own assignment. If you discussed the homework or solutions with others, a list of collaborators must be included with each submission. Each of the collaborators has to write the solutions in their own words (copies are not allowed). Also, recall the AI policy:

- You may **not use AI tools to generate complete solutions** to assignments, coding problems, or any assessed work that is intended to measure your independent knowledge and skills.
- Whenever you use AI tools to assist in your assignment, you must **include a paragraph at the end of the assignment explaining what you used the AI for and include all the prompts you used to get the results. Failure to do so is in violation of the academic honesty policies**.

General Requirements

- When proofs are required, do your best to make them both clear and rigorous. Even when proofs are not required, you should justify your answers and explain your work.
- When asked to present a construction, you should show the correctness of the construction.

Some Useful Equalities

- $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
- $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$
- $2^{\log_2 n} = n$, $a^{\log_b n} = n^{\log_b a}$, $n^{n/2} \leq n! \leq n^n$, $\log x^a = a \log x$
- $\log(a \times b) = \log a + \log b$, $\log(a/b) = \log a - \log b$
- $a + ar + ar^2 + ... + ar^{n-1} = \frac{a(r^n-1)}{r-1}$
- $1 + \frac{1}{2} + \frac{1}{2^2} + ... + \frac{1}{2^n} = 2(1 - \frac{1}{2^{n+1}})$
- $1 + 2 + 4 + ... + 2^n = 2^{n+1} - 1$

1. **(5 pts)** Let $f(n) = 2n^2$ and $g(n) = n^2$. Prove or disprove the following statements.

   (a) **(1 pts)** $f(n) \leq cg(n)$ when $c = 2$ and $n = 1$

   To prove: $f(n) \leq cg(n)$ when $c = 2$ and $n = 1$.
   With $c = 2, n = 1$, we get $f(n) = 2(1)^2 = 2$ and $cg(n) = 2(1)^2 = 2$. So $f(n) = cg(n)$ and as a result, $f(n) \leq cg(n)$.

   (b) **(2 pts)** $f(n) \leq cg(n), \forall n \geq n_0$ when $c = 2$ and $n_0 = 1$.

   To prove: $f(n) \leq cg(n), \forall n \geq n_0$ when $c = 2$ and $n_0 = 1$.
   With $c = 2$, we get $cg(n) = 2n^2 = f(n)$ for all n. Thus, we can conclude that $f(n) = cg(n)$ for all $n \geq n_0$, which implies that $f(n) \leq cg(n)$ for all $n \geq n_0$.

   (c) **(2 pts)** For all natural numbers $c > 0$, there exists $n_0 > 0$ such that for all $n \geq n_0$, $f(n) \leq cg(n)$.

   We will disprove the statement by showing that there exists $c > 0$ such that for all $n_0 > 0$, there exists $n \geq n_0$ such that $f(n) > cg(n)$.
   Consider $c = 1$. Then, $cg(n) = n^2 < f(n)$ for all $n \geq 0$. Thus, given an arbitrary $n_0 > 0$, we can pick $n = n_0$, which results in $f(n) > cg(n)$.

2. **(15 pts)** Let $f(n) = 2^n$ and $g(n) = n^8$. Prove or disprove the following statements.

(a) **(1 pts)** $f(n) \leq g(n)$ when $n = 5$.

To prove: $f(5) \leq g(5)$.

$$\begin{aligned} f(5) &= 2^5 = 32 \\ g(5) &= 5^8 \geq 5^3 = 125 \geq 32 \end{aligned}$$

Thus, $g(5) \geq f(5)$.

(b) **(2 pts)** $f(n) \leq cg(n), \forall c \geq 1$ when $n = 5$.

To prove: $f(5) \leq cg(5), \forall c \geq 1$.
From part i, we get that $g(5) \geq 0$. Thus,

$$cg(5) \geq g(5), \forall c \geq 1$$

Also, from part i, we have $g(5) \geq f(5)$. Combining this with the above inequality, we get $cg(5) \geq g(5) \geq f(5), \forall c \geq 1$.

(c) **(2 pts)** $f(n) > g(n)$ when $n = 2^6$.

To prove: $f(2^6) > g(2^6)$.
First, we know that $2^6 = 64 > 48$. Since 2^n is an increasing function, we get

$$2^{(2^6)} > 2^{48} \tag{1}$$

But $2^{48} = 2^{6\times 8} = (2^6)^8$. Combining this with the Eq (1), we get $2^{(2^6)} > (2^6)^8$. Thus, $f(2^6) > g(2^6)$.

(d) **(4 pts)** There exists $n_0 > 0$ such that $f(n) > g(n), \forall n \geq n_0$.

To prove: With $n_0 = 2^6$, we get $f(n) > g(n), \forall n \geq n_0$.

First, we will show that the ratio $f(n)/g(n)$ increases as n increases. Let $h(n) = f(n)/g(n)$. We will show that $h(n+1) > h(n), \forall n \geq 2^6$

$$\begin{aligned}
\frac{h(n+1)}{h(n)} &= \frac{f(n+1)g(n)}{f(n)g(n+1)} \\
&= \frac{2^{n+1}n^8}{2^n(n+1)^8} \\
&= 2\left(\frac{n}{n+1}\right)^8 \\
>= \; & 2\left(\frac{2^6}{2^6+1}\right)^8, \forall n \geq 2^6 \text{ since } \frac{n}{n+1} \text{ increases as } n \text{ increases} \\
> \; & 1, \forall n \geq 2^6
\end{aligned}$$

Since $f(n), g(n) > 0, \forall n \geq 2^6$, we can conclude that $h(n) > 0, \forall n \geq 2^6$. We can therefore multiply both sides of the above inequality by $h(n)$ to obtain

$$h(n+1) > h(n), \forall n \geq 2^6. \tag{2}$$

From part (c), we get that $h(n) > 1$ when $n = 2^6$. Combining this with Eq (2), we get

$$h(n) = \frac{f(n)}{g(n)} > 1, \forall n \geq 2^6.$$

Since $g(n) > 0$ for all $n \geq 2^6$, we can conclude that $f(n) > g(n), \forall n \geq 2^6$.

(e) **(1 pts)** $g(n) \in O(f(n))$. You must use the result from part (d).

To prove: $g(n) \in O(f(n))$.

From part (d), we proved that with $n_0 = 2^6$ and $c = 1$, $g(n) \leq f(n)$. Thus, by definition, $g(n) \in O(f(n))$.

(f) **(5 pts)** $f(n) \in O(g(n))$.

To prove: $f(n) \notin O(g(n))$.

Assume the opposite, i.e., $f(n) \in O(g(n))$. Then, by definition, there exists $c > 0$, $n_0 > 0$ such that for all $n \geq n_0$

$$\begin{aligned} 2^n &\leq cn^8 \\ n &\leq 8c \log n \\ \frac{n}{\log n} &\leq 8c \end{aligned}$$

Applying L'Hospital's Rule as we did in class, we get

$$\lim_{n\to\infty} \frac{n}{\log n} = \lim_{n\to\infty} \frac{\frac{d}{dn} n}{\frac{d}{dn} \log n} = \lim_{n\to\infty} \frac{1}{\frac{1}{n}} = \lim_{n\to\infty} n$$

So there is no constants c and n_0 such that for all $n \geq n_0$,

$$\frac{n}{\log n} \leq 8c$$

This leads to a contradiction. Hence, our assumption is false and $f(n) \notin O(g(n))$.

3. **(5 pts)** Prove or disprove: Any function that is in $O(\log_2 n)$ is also in $O(\log_3 n)$.

   To prove: Any function $f(n)$ that is in $O(\log_2 n)$ is also in $O(\log_3 n)$.

   Proceeding from the antecedent

$$\begin{aligned}
& \exists c > 0.\ \exists n_0 > 0.\ \forall n \geq n_0. && f(n) \leq c \times \log_2 n \\
\Rightarrow\ & \exists c > 0.\ \exists n_0 > 0.\ \forall n \geq n_0. && f(n) \leq c \times \frac{\log_3 n}{\log_3 2} \quad \text{changing the log base} \\
\Rightarrow\ & \exists c' > 0.\ \exists n_0 > 0.\ \forall n \geq n_0. && f(n) \leq c' \times \log_3 n \\
& && \qquad \text{where } c' = \frac{c}{\log_3 2} \text{ for some } c \\
\Rightarrow\ & f(n) \in O(\log_3 n)
\end{aligned}$$

4. **(5 pts)** Prove or disprove: If $f(n) \leq 100g(n)$ for infinitely many n, then $f(n) \in O(g(n))$.

   We will disprove the statement.

   Consider

$$\begin{aligned}
f(n) &= \begin{cases} n & \text{if n is even} \\ n^2 & \text{if n is odd} \end{cases} \\
g(n) &= n
\end{aligned}$$

   Then, for all even n, $f(n) \leq 100g(n)$. Since there are infinitely many even numbers, $f(n) \leq 100g(n)$ for infinitely many n.

   However, there does not exist constants $c, n_0 > 0$ such that $f(n) \leq cg(n)$ for all $n \leq n_0$. To see this, suppose such c and n_0 exists. Then, it must be the case that for all odd $n \geq n_0$, $f(n) \leq cg(n)$. Thus,

$$\begin{aligned}
n^2 &\leq cn \\
n &\leq c
\end{aligned}$$

   This leads to a contradiction because there is no constant c such that $c \geq n$ for all odd $n \geq n_0$. Thus, we can conclude that $f(n) \notin O(g(n))$.

5. **(5 pts)** Consider arbitrary functions $f, g : \mathbb{N} \to \mathbb{N}$. For each of the following statements, determine whether it provides **sufficient information** to determine whether $f(n) \in O(g(n))$. Briefly justify your answer in each case.

   (a) **(1 pts)** $f(n) > 2g(n)$ for all $n \geq 1$.

   We **cannot** determine whether $f(n) \in O(g(n))$ based on this statement. Even if $f(n) > cg(n), \forall n \geq n_0$ for specific values of c and n_0, we do not know whether there are other values of c and n_0 such that $f(n) \leq cg(n), \forall n \geq n_0$.

   (b) **(1 pts)** For all natural numbers $c > 0$, there exists $n_0 > 0$ such that for all $n \geq n_0$, $f(n) \leq cg(n)$.

   We **can** conclude that $f(n) \in O(g(n))$ based on this statement. We can pick an arbitrary $c > 0$. The above statement ensures that there exists $n_0 > 0$ such that $f(n) \leq cg(n), \forall n \geq n_0$ and by definition $f(n) \in O(g(n))$.

   (c) **(1 pts)** There exists $n_0 > 0$ such that $f(n) \leq g(n), \forall n \geq n_0$.

   We **can** conclude that $f(n) \in O(g(n))$ based on this statement. We can pick $c = 1$. The above statement ensures that there exists $n_0 > 0$ such that $f(n) \leq cg(n), \forall n \geq n_0$ and by definition $f(n) \in O(g(n))$.

   (d) **(1 pts)** $f(n) > cg(n)$ for all $c > 0$ and $n > 0$.

   We **can** conclude that $f(n) \notin O(g(n))$ based on this statement because it implies that there does not exist any $c > 0$ and $n_0 > 0$ such that $f(n) \leq cg(n), \forall n \geq n_0$. Thus, by definition, $f(n) \notin O(g(n))$.

   (e) **(1 pts)** $g(n) \in O(f(n))$.

   We **cannot** determine whether $f(n) \in O(g(n))$ based on this statement. For example, if $f(n) = g(n)$, then $g(n) \in O(f(n))$ and $f(n) \in O(g(n))$. However, with $f(n) = 2^n$ and $g(n) = n^8$, we showed in Problem 2 that $g(n) \in O(f(n))$ while $f(n) \notin O(g(n))$.

6. **(15 pts)** Consider the following algorithm:

```
GCD(a, b) {
  x = max(a, b)
  y = min(a, b)
  while (y != 0) {
    z = x % y;
    x = y;
    y = z;
  }
  return x;
}
```

Let x_i and y_i represent the values of the variables `x` and `y` at the end of the i-th iteration of the while loop. Let x_0 and y_0 be the initial values of `x` and `y` before entering the loop.

(a) **(6 pts)** Complete the following table, expressing each cell **only in terms of x_0, y_0, and constants** such that the upper bound on y_6 is strictly less than y_0. For this part, assume that the values of a and b are such that the loop runs for at least 6 iterations.

Hint: Use the following properties of modulus (%) operation:

- For any pair of numbers u, v,

$$u \geq v \implies u\%v \leq \frac{u}{2}. \tag{3}$$

- For any pair of numbers u, v,

$$u\%v < v \tag{4}$$

i	Upper bound on x_i	Upper bound on y_i
1	y_0	$\frac{x_0}{2}$
2	$y_1 \leq \frac{x_0}{2}$	$\frac{x_1}{2} \leq \frac{y_0}{2}$
3	$y_2 \leq \frac{y_0}{2}$	$\frac{x_2}{2} \leq \frac{x_0}{2^2}$
4	$y_3 \leq \frac{x_0}{2^2}$	$\frac{x_3}{2} \leq \frac{y_0}{2^2}$
5	$y_4 \leq \frac{y_0}{2^2}$	$\frac{x_4}{2} \leq \frac{x_0}{2^3}$
6	$y_5 \leq \frac{x_0}{2^3}$	$\frac{x_4}{2} \leq \frac{y_0}{2^3}$

First iteration: Since $x_0 = \max(a, b)$ and $y_0 = \min(a, b)$, we can conclude that $x_0 \geq y_0$. Thus, from (3), we obtain

$$z = \ x_0 \% y_0 \ \leq \frac{x_0}{2}.$$

Additionally, from (4), we have $z < y_0$. Thus, at the end of the first iteration, we get

$$x_1 = y_0, \ y_1 \leq \frac{x_0}{2}, \text{ and } y_1 < x_1.$$

Second iteration: Since $x_1 > y_1$, applying (3) again gives

$$z = \ x_1 \% y_1 \ \leq \frac{x_1}{2}.$$

Similarly, from (4), $z < y_1$. Thus, at the end of the second iteration, we get

$$x_2 = y_1, \ y_2 \leq \frac{x_1}{2}, \text{ and } y_2 < x_2.$$

General pattern for iteration i: By continuing this process, we establish that for each iteration $i \geq 1$:

$$x_i = y_{i-1}, \ y_i \leq \frac{x_{i-1}}{2}, \text{ and } y_i < x_i$$

Thus, for each row i, we can use the values of x_{i-1} and y_{i-1} from the previous row $(i-1)$ to complete the table as shown above.

(b) **(2 pts)** Consider iteration $i = 2j$ for some $j \in \{1, 2, \ldots\}$, i.e., i is even. Express the upper bounds on x_i and y_i in terms of x_0, y_0, and j.

From the previous part, we get that

$$\begin{aligned} x_i &= y_{i-1} \leq \frac{x_0}{2^j} \\ y_i &\leq \frac{x_{i-1}}{2} \leq \frac{y_0}{2^j} \end{aligned}$$

(c) **(4 pts)** What is the maximum number of iterations that the while loop can execute before terminating? Provide the **upper bound** on the number of iterations in terms of x_0 and y_0, **including all constant factors**. Do not simply provide an asymptotic (Big-O) bound.

From the previous part, we established that at iteration $i = 2j$,

$$y_i \leq \frac{y_0}{2^j}$$

This expression reaches 1 when $j = \log_2 y_0$. Since $i = 2j$, the total number of iterations before $y_i = 1$ is at most $2 \log_2 y_0$.

At the next iteration $(i + 1)$, we have $y_{i+1} = 0$, which causes the while loop to terminate. Thus, the maximum number of iterations the loop can execute is

$$2 \log_2 y_0 + 1.$$

(d) **(3 pts)** The input size, denoted by n, is the number of bits required to represent the inputs, i.e., $a, b \leq 2^n$. Derive the Big-O upper bound on the runtime of `GCD` as a function of n required to represent a and b).

Note: If you were unable to complete the previous part, you may assume that the while loop runs $O(\log_2(y_0))$ times in the worst case.

From the previous part, we established that the while loop runs $O(\log y_0)$ times.

Since $y_0 \leq a \leq 2^n$, we can conclude that the loop runs at most $O(\log(2^n))$, which is $O(n)$ times. Each iteration takes constant time.

So the total time complexity is $O(n)$.